

Cardiola — Handoff Backend → Frontend

Fase 7 · Plataforma de Inteligência Cardíaca Total

Contrato e guia de uso da API · gerado em 2026-05-29

0. Como usar este documento

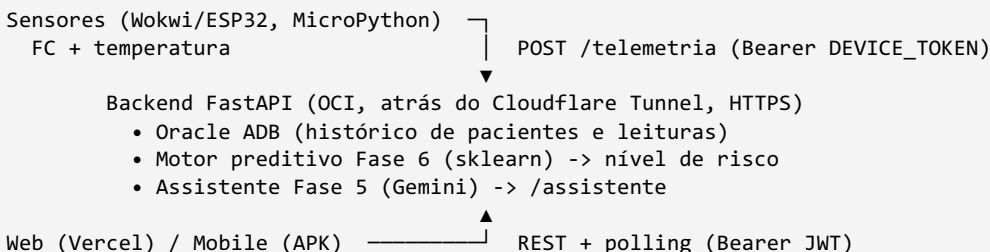
Este documento descreve tudo que o backend da Cardiola expõe e como consumi-lo a partir dos frontends (Web React+Vite e Mobile React Native+Expo). Foi escrito para dois públicos:

- **Pessoas do time de frontend** — leitura humana, com exemplos prontos.
- **Agentes LLM** — pode ser colado como contexto para gerar/ajudar o código do app; os formatos JSON e nomes de campos são explícitos.

Fonte da verdade em caso de dúvida: o **Swagger interativo** em <https://college-anonymous-thesaurus-assumption.trycloudflare.com/docs> e o arquivo backend/README.md no repositório.

1. Visão geral e arquitetura

O backend é o núcleo integrador (FastAPI, em Python). Ele recebe a telemetria dos sensores, classifica o risco cardíaco com um modelo preditivo (Fase 6), responde um assistente conversacional via Gemini (Fase 5) e expõe tudo às interfaces por uma API REST com autenticação bearer. Persistência em Oracle Autonomous Database; o serviço fica exposto por HTTPS através de um Cloudflare Tunnel.



Idioma: toda a API é em português (rotas, campos JSON e tabelas do banco).

2. Base URL e ambiente

URL pública atual (snapshot): <https://college-anonymous-thesaurus-assumption.trycloudflare.com>

⚠ A URL é **efêmera** (Cloudflare quick tunnel, sem domínio): muda quando o túnel reinicia. Não fixe no código — leia de uma config/variável de ambiente do app. Para descobrir a URL vigente, o responsável pelo backend roda:

```
ssh oracle-large-br "journalctl -u cloudflared-quick | grep -Eo 'https://[a-z0-9-]+\.\trycloudflare\.com' | tail -1"
```

- **Swagger / OpenAPI:** <https://college-anonymous-thesaurus-assumption.trycloudflare.com/docs> e <https://college-anonymous-thesaurus-assumption.trycloudflare.com/openapi.json>
- **Healthcheck:** GET <https://college-anonymous-thesaurus-assumption.trycloudflare.com/health> → {"status":"ok",...}
- Todas as respostas são JSON (UTF-8). Envie Content-Type: application/json nos POST/PUT.

3. Autenticação

Não há tabela de usuários — é proposital para o escopo (autenticação simples). Existe um único usuário demo definido por variáveis de ambiente no servidor:

Usuário	Senha
medico	cardioia123

Fluxo:

- 1. O frontend faz POST /autenticacao/login com { "usuario": "...", "senha": "..." }.
- 2. Em caso de sucesso, recebe um **JWT** (HS256, validade ~12 h) no campo token_acesso.
- 3. O frontend envia esse token em **todas** as rotas protegidas no header Authorization: Bearer <token_acesso>.
- 4. O backend valida assinatura + expiração a cada request (stateless, sem consulta ao banco). Token inválido/expirado → **401** (o front limpa o token e volta ao login).

Exemplo:

```
curl -s https://college-anonymous-thesaurus-assumption.trycloudflare.com/autenticacao/login \
-H 'Content-Type: application/json' \
-d '{"usuario":"medico","senha":"cardioia123"}'

# resposta:
{ "token_acesso": "eyJhbGciOiJIUzI1NiIsInR5cGU": "bearer" }
```

Importante: o endpoint de ingestão /telemetria usa OUTRO token (o DEVICE_TOKEN estático, do dispositivo IoT/emulador). O frontend **nunca** usa o DEVICE_TOKEN — só o JWT do login.

4. Convenções gerais

- Formato:** JSON em UTF-8; Content-Type: application/json nos corpos.
- Datas:** ISO-8601 em UTC (ex.: "2026-05-29T02:36:51Z") no campo momento. Converta para o fuso local na UI.
- Risco:** nivel_risco ∈ baixo | medio | alto; pontuacao_risco é float 0..1 (multiplique por 100 para %).
- Erros:** formato do FastAPI { "detail": "..." }. Códigos: 200/201/204 ok; 401 sem/má auth; 404 não encontrado; 422 corpo inválido; 500 erro interno.
- CORS:** liberado (será restringido ao domínio da Vercel em produção). Auth por header bearer, sem cookies.

5. Referência de endpoints

Resumo (auth: JWT = token do login; DEVICE = DEVICE_TOKEN do dispositivo):

Método	Rota	Auth	Corpo / Query	Resposta
POST	/autenticacao/login	—	{usuario, senha}	{token_acesso, tipo_token}
GET	/pacientes	JWT	—	[{paciente, ultima}]
POST	/pacientes	JWT	{nome, idade, sexo?, observacoes?}	201 PacienteSaida
PUT	/pacientes/{id}	JWT	{nome, idade, sexo, observacoes}	PacienteSaida
DELETE	/pacientes/{id}	JWT	—	204

Método	Rota	Auth	Corpo / Query	Resposta
GET	/pacientes/{id}/ultima	JWT	–	LeituraSaida (404 se vazio)
GET	/pacientes/{id}/historico	JWT	?limite=100	[LeituraSaida] (crescente)
GET	/alertas	JWT	–	[{paciente, leitura}]
POST	/telemetria	DEVICE	{paciente_id, fc, temperatura, momento?}	201 LeituraSaida
POST	/assistente	JWT	{mensagem, paciente_id?, conversa_id?}	{resposta, modelo, contextualizado, conversa_id}
DELETE	/assistente/{conversa_id}	JWT	–	204
GET	/emulador	JWT	–	EmuladorStatus
POST	/emulador/ligar	JWT	{pacientes?, intervalo?}	EmuladorStatus
POST	/emulador/desligar	JWT	–	EmuladorStatus
GET	/health	—	–	{status, service}

6. Modelos de dados

PacienteSaida

Campo	Tipo	Descrição
id	int	Identificador do paciente
nome	string	Nome do paciente
idade	int	Idade em anos
sexo	string	Ex.: "F", "M", "?"
observacoes	string	Notas/histórico (pode ser vazio)

LeituraSaida

Campo	Tipo	Descrição
id	int	Identificador da leitura
paciente_id	int	FK para o paciente
momento	string (ISO-8601 UTC)	Instante da leitura
fc	float	Frequência cardíaca (bpm)
temperatura	float	Temperatura (°C)
pontuacao_risco	float (0..1)	Probabilidade de risco do modelo
nivel_risco	string	baixo medio alto
recomendacao	string	Recomendação clínica gerada

Compostos e demais

- **PacienteStatus** = { paciente: PacienteSaida, ultima: LeituraSaida | null } (usado em GET /pacientes; ultima é null quando o paciente ainda não tem leitura).
- **AlertaSaida** = { paciente: PacienteSaida, leitura: LeituraSaida }
- **RespostaChat** = { resposta, modelo, contextualizado, conversa_id }
- **EmuladorStatus** = { ligado, pacientes, intervalo, leituras_geradas }

7. Padrões de uso no frontend

7.1 Login e sessão

- Tela de login (usuário + senha) → POST /autenticacao/login → guardar token_acesso (web: memória/localStorage; mobile: SecureStore).
- Interceptor HTTP anexa Authorization: Bearer <token> em toda chamada; em 401, limpar token e voltar ao login.

7.2 Dashboard (lista de pacientes)

- Polling de GET /pacientes a cada ~3–5 s. Cada item traz o paciente e a ultima leitura (ou null).

```
TOKEN=... # do login
curl -s https://college-anonymous-thesaurus-assumption.trycloudflare.com/pacientes -H
"Authorization: Bearer $TOKEN"

# resposta (exemplo):
[ { "paciente": { "id":1,"nome":"Maria Silva","idade":67,"sexo":"F","observacoes":"..."},
  "ultima": { "id":12,"paciente_id":1,"momento":"2026-05-29T02:36:51Z","fc":150,
    "temperatura":39.4,"pontuacao_risco":0.99,"nivel_risco":"alto",
    "recomendacao":"Risco elevado..." } },
  { "paciente": { "id":2,...}, "ultima": null } ]
```

7.3 Detalhe do paciente

- GET /pacientes/{id}/ultima para o card de sinais atuais (404 se ainda não há leitura).
- GET /pacientes/{id}/historico?limite=100 para o gráfico (ordem cronológica crescente; [] se vazio).

7.4 Painel de alertas

- GET /alertas → pacientes cuja última leitura está em risco medio ou alto (ordenados por gravidade).

7.5 Cadastro/edição de pacientes (CRUD)

- Criar: POST /pacientes {nome, idade, sexo?, observacoes?}. Editar: PUT /pacientes/{id}. Remover: DELETE /pacientes/{id} (apaga também as leituras do paciente).

7.6 Assistente (chat multi-turno)

- 1ª mensagem **sem** conversa_id → o backend cria a conversa e devolve conversa_id; guarde-o.
- Mensagens seguintes **com** o mesmo conversa_id → o assistente lembra do contexto.
- Botão "nova conversa": DELETE /assistente/{conversa_id} apaga o histórico.
- Passe o paciente_id selecionado para respostas contextualizadas. Mostre estado de "digitando" (chamada ~1–3 s). Se modelo vier "fallback"/"error", indique indisponibilidade.

```
# 1ª mensagem (guarde o conversa_id devolvido)
curl -s https://college-anonymous-thesaurus-assumption.trycloudflare.com/assistente -H
"Authorization: Bearer $TOKEN" \
-H 'Content-Type: application/json' \
-d '{"mensagem":"Como está a paciente 1?","paciente_id":1}'
# -> { "resposta":"...", "modelo":"gemini-3.1-flash-lite",
#     "contextualizado":true, "conversa_id":"8ff9c6ab..." }

# 2ª mensagem na MESMA conversa
curl -s https://college-anonymous-thesaurus-assumption.trycloudflare.com/assistente -H
"Authorization: Bearer $TOKEN" \
-H 'Content-Type: application/json' \
-d '{"mensagem":"E o que devo fazer?","conversa_id":"8ff9c6ab..."}'
```

```
# nova conversa (apaga histórico)
curl -s -X DELETE https://college-anonymous-thesaurus-
assumption.trycloudflare.com/assistente/8ff9c6ab... -H "Authorization: Bearer $TOKEN"
```

Nota: o histórico fica em memória no servidor (zera se o backend reiniciar) e guarda as últimas 20 mensagens por conversa.

7.7 Emulador (simulação ao vivo, controlada pela UI)

- Toggle "Simulação ao vivo": POST /emulador/ligar {pacientes:[1,2,3], intervalo:3} para começar a gerar leituras; POST /emulador/desligar para parar; GET /emulador para o estado.
- Enquanto ligado, grava leituras reais no banco (o dashboard fica vivo via polling). **Lembre de desligar** ao terminar.

8. Estados vazios e dinâmica dos dados

- Pacientes iniciam **sem leituras**: ultima = null em /pacientes; /ultima → 404; /historico → []. A UI deve tratar ("sem dados").
- Cada device (Wokwi) representa **um** paciente (um wearable por pessoa). Só o paciente alimentado recebe leituras novas; os demais ficam estáticos até serem alimentados.
- Para o dashboard "vivo" com vários pacientes, use o **emulador** (multi-paciente) — via API (seção 7.7) ou o script CLI backend/emulator/emulate.py.

9. Playbook da demonstração

1. Login na UI (medico / cardioia123).
2. Ligar a simulação ao vivo (POST /emulador/ligar com pacientes [1,2,3]) — o dashboard começa a atualizar e alertas aparecem.
3. (Opcional) Rodar o Wokwi para o paciente 1 em paralelo, mostrando o hardware (sensores + OLED/LED) fechando o fluxo fim-a-fim.
4. Navegar: lista → detalhe (gráfico do histórico) → painel de alertas → conversar com o assistente sobre um paciente.
5. Ao final, desligar o emulador (POST /emulador/desligar).

10. Limitações e observações

- URL pública efêmera (quick tunnel) — confirme a vigente antes da demo; para URL fixa, registrar um domínio (named tunnel).
- Histórico do chat em memória (processo único) — some no restart.
- Um único usuário demo (sem cadastro/roles). Senha demo fraca — não é para exposição pública prolongada.
- O emulador grava dados reais no banco enquanto ligado — desligar ao terminar.

Apêndice — fluxo curl completo

```
BASE=https://college-anonymous-thesaurus-assumption.trycloudflare.com
TOKEN=$(curl -s $BASE/autenticacao/login -H 'Content-Type: application/json' \
  -d '{"usuario":"medico","senha":"cardioia123"}' | jq -r .token_acesso)

# lista (polling do dashboard)
curl -s $BASE/pacientes -H "Authorization: Bearer $TOKEN" | jq
# detalhe
curl -s $BASE/pacientes/1/ultima -H "Authorization: Bearer $TOKEN" | jq
```

```
curl -s $BASE/pacientes/1/historico -H "Authorization: Bearer $TOKEN" | jq
# alertas
curl -s $BASE/alertas -H "Authorization: Bearer $TOKEN" | jq
# assistente
curl -s $BASE/assistente -H "Authorization: Bearer $TOKEN" \
  -H 'Content-Type: application/json' -d '{"mensagem":"Resuma o quadro","paciente_id":1}' | jq
# simulação ao vivo
curl -s -X POST $BASE/emulador/ligar -H "Authorization: Bearer $TOKEN" \
  -H 'Content-Type: application/json' -d '{"pacientes":[1,2,3],"intervalo":3}' | jq
curl -s -X POST $BASE/emulador/desligar -H "Authorization: Bearer $TOKEN" | jq
```